

Werner Bisschoff

HYBRID EDGE/SYSTEMS ENGINEER

AWS Certified Solutions Architect – Associate (In Progress — Target: 6 Weeks)

loc: Cape Town, Western Cape, South Africa

tel: 071 826 2066 email: werner@bisschoff.dev github: wbisschoff13

linkedin: linkedin.com/in/wbisschoff13 web: https://werner.bisschoff.dev

Hybrid Edge/Systems Engineer with 5+ years of experience architecting end-to-end telemetry systems spanning edge compute to backend integration. Specializes in C/C++ and FreeRTOS for deterministic real-time scheduling, Python for automation and hardware-in-the-loop validation, and Elixir/OTP for distributed platform infrastructure.

Skills

Primary Competencies : C, C++, FreeRTOS, ESP32/ESP-IDF, NFC APDU (ISO 14443-4)

Secondary Competencies : NimBLE, Bare-Metal Runtimes, UART/SPI/I2C, Hardware-in-the-Loop (HIL) Testing

Foundational Systems : Linux IPC, Bash, Python (pytest, Custom Automation Frameworks), Assembly, System Modeling

Cross-Domain Integration : ERPNext (Integration-focused), Docker, Spec-Driven Development (SDD)

Experience

Principal Systems Architect

Cape Town

DIVERGENT TABLETOP

Jul 2025

- Engineered a distributed real-time data engine utilizing Elixir/OTP and conflict-free replicated data types (CRDT models). Deployed in-memory GenServer storage buffers with threshold-based flush mechanics, reducing direct database write transactions by approximately 85% under sustained edit workloads.
- Integrated a Rust-backed CRDT synchronization layer (y_ex) using actor-based concurrency, achieving sub-50ms convergence latency across distributed editing sessions through deterministic state reconciliation and binary delta serialization.
- Designed a dynamic BEAM supervision tree with one_for_one isolation, encapsulating each page editing session in its own process subtree to prevent cascading failures and guarantee per-session state integrity.

Embedded Systems & Integration Engineer

Cape Town

FARO AFRICA

Aug 2024 – Nov 2025

- Designed ISO 14443-4 NFC communication architectures for e-paper display nodes and secure card reader matrices.
- Engineered lock-free asynchronous device management services bridging hardware peripherals with application layers.
- Built Python-based hardware peripheral simulators to catch edge-case device state corruptions prior to flashing.
- Introduced LLM-assisted development workflows to improve debugging speed and code review throughput.

Firmware Engineer (Contract)

Remote

INGENICS DIGITAL GMBH (THROUGH VIVA OUTSOURCING)

Mar 2023 – May 2024

- Architected deterministic FSMs on dual-core ESP32 using FreeRTOS for concurrent sensor arrays.
- Developed Cap'n Proto IPC over serial lines, achieving sub-millisecond serialization latency.

Software Developer

Cape Town

UMAN TECHNOLOGIES

Mar 2021 – Dec 2022

- Used perf profiling and flamegraphs to trace a performance bottleneck to JSON serialization and verbose logging in a SOME/IP request handling loop. Eliminated unnecessary logging via config toggle reducing latency by approximately 80%, then drove an architecture redesign to phase out JSON entirely.

Our C++ library for customer hardware had severe latency issues — customers couldn't use the software at all.

I was tasked to investigate and fix the performance bottleneck.

I used perf to profile the hot paths during live requests, generated flamegraphs, and traced the problem to JSON serialization in the request handling loop. Verbose logging on every JSON operation made it worse. I disabled the unnecessary logging via a config toggle as a first step. I then participated in the design discussion and helped drive the effort to phase out JSON entirely through a complete architecture redesign for dynamic payload creation. I self-taught the SOME/IP protocol from specs to ensure the new approach was spec-compliant.

Removing verbose logging alone improved some request calls by 80%. The full architecture redesign eliminated the bottleneck long-term, restoring full functionality for customers.

- Sole owner of an IPC service layer — studied existing node schema, designed node structures and data contracts, and implemented the communication layer enabling the frontend to consume the service.

Customers needed to dynamically add our service to the hardware box and interact with it via IPC.

Design and implement the IPC layer for our SOME/IP simulation service — specifically, what nodes to expose and what data they would contain — so the frontend could consume it.

I was the sole owner of our service's IPC implementation. I studied the existing node schema and designed how to expose our service, defining the node structure and data contracts. There were a few iterations early on — I had some initial misunderstandings about how it should work — but I worked through them and wrote the code to make our service available to the frontend.

The IPC layer was implemented and customers could successfully integrate our service into the hardware box.

- Built Python-based SOME/IP hardware mocks simulating request/response behavior; integrated into CI so automated tests ran without physical hardware.

Physical hardware wasn't always available for testing. Automated tests couldn't run at all.

Enable automated testing without access to real hardware.

I built Python mocks that simulated SOME/IP requests and responses, creating a test harness that replicated hardware behavior. I ensured the mocks were integrated into CI so tests ran automatically against them.

The mocks were adopted by the team and integrated into CI. Automated tests could run without physical hardware, improving test coverage and developer productivity.

Education

B.Eng. Computer and Electronic Engineering

NORTH-WEST UNIVERSITY

2020

Projects

Divergent Tabletop Wiki

- Built a community wiki using Astro, Elixir, and Docker for knowledge management. Documented event frameworks, onboarding processes, and communication best practices. Created tooling for content management and community operations.

Ingenics Digital GmbH — Event-Driven FSM for Embedded Systems

- Designed event-driven finite state machine for I2C-based embedded system using C++ and FreeRTOS. Solved complex state management challenges in real-time embedded environment. Outcome: Maintainable in-house architecture leading to fewer bugs and quicker development cycles.